

Bloom Filter and Hashing Analysis

Ryan Sargis
CMPSC 130A
Professor Vigoda

November 25, 2025

1 Introduction

For this project, I tested two different hash functions and utilized them to implement a bloom filter. The two hash functions that I used were a universal hash function (hash 1) and a seeded pseudo-random hash function (hash 2). Using these hash functions, my first goal was to compare their performance for both random data from the universe and uniform data. Then, after testing the hash functions, the next goal was to implement a bloom filter using both types of hash functions. Finally, once both bloom filters were implemented, I tested them using varying c and k values to generate a plot of false positive rates that could be compared to the equation for the theoretical false positive rate of a bloom filter.

2 Methods

2.1 Hash Functions

The two equations for the hash functions were $h_i(x) = ((a_i x + b_i) \bmod p) \bmod m$ and $h_i(x) = r(s_i + x)$. Where a_i, b_i , and s_i are randomly generated constants. To generate these random integers, I used the Mersenne twister and uniform int distribution from the C++ random library. These constants and the two hash functions were stored in a class that I created to help me gather the hashing data.

For my parameters, I selected the size of the universe (N) and the prime number (p) to be $2^{31} - 1$. I chose this value since universal hashing calls for $p \geq N$, and I felt that $2^{31} - 1$ was a sufficiently large universe size since it is the largest Mersenne prime that fits in a C++ unsigned int. To gather my hashing data, I chose to hash $n = 1000$ values with $c = 5$. I felt that $n = 1000$ was sufficiently large to test the effectiveness of each hash function while still being small enough to effectively observe in a scatter plot. With c and n fixed, the m used in the universal hash function was also fixed to cn .

To gather my data, I simply used each hash function to hash the 1000 values while keeping track of the each bin's load to eventually find the maximum load. With both randomly generated and linear (numbers 0-999) inputs, I ran 10 trials to get the average maximum bin load for each hash function and set of inputs.

2.2 Bloom Filter

To implement the bloom filter, I created a BloomFilter class which utilizes the two different hash functions to implement Add(x) and Contains(x). The constants a_i, b_i , and s_i used in the hash functions are randomly generated

when the constructor is called and are added to vectors of size k .

For parameters, p and N remained the same since the value $2^{31} - 1$ is still sufficient for implementing the bloom filter. For the constant c , I chose 5, 10, and 15 since I felt that those values would provide good variety of false positive rates to analyze. For each c value, I tested 5 different k values since I felt that having 5 k values would be sufficient to create a curve to compare with the theoretical false positive rate curve. For the k values, I used the theoretical optimal value $c \ln(2)$ rounded to the nearest integer, as well as $c \ln(2) \pm 1$ and $c \ln(2) \pm 2$ for each value of c . For n , I was initially using $n = 10,000$ to ensure quick data generation when running my code. However, while testing, using $n = 10,000$ caused too much variability when the false positive rate was very small, so I opted to switch to $n = 100,000$ to fix this issue. Like before, since n and c are fixed, m is also fixed at $m = cn$.

To gather data, I first created two bloom filter objects so that I could insert to each of them using the two hash functions. In each bloom filter, I inserted n random values to each bloom filter and kept track of these values as the ground truth. Then, I queried n random values and kept track of the number of false positives by comparing any values reported as found to my ground truth. This process was done for all 15 different combinations of c and k , and repeated 10 times to get median false positive rate values to analyze. The program was recompiled between each trial so that new random constants for a_i , b_i and s_i were generated.

3 Results

3.1 Hash Functions

When hashing random inputs, the two hash functions produced similar results that were close to what I had expected. The data gathered from both functions produced a random-looking scatter plot, which is desirable for hash functions. For the universal hash function, the average maximum bin load over 10 trials was 3.2 whereas the average for the seeded pseudo-random hash function was 3.5.

When tested with linear data, however, something very interesting occurred. As can be seen in Figure 1, hashing linear values 0 to 999 with the universal hash function resulted in a correlated scatter plot as opposed to random. The scatter plot for the seeded pseudo-random hash function still appeared random-looking as it did for random data. The average maximum bin load for the universal hash function was 1.4 while the average for the seeded pseudo-random hash function was 3.7.

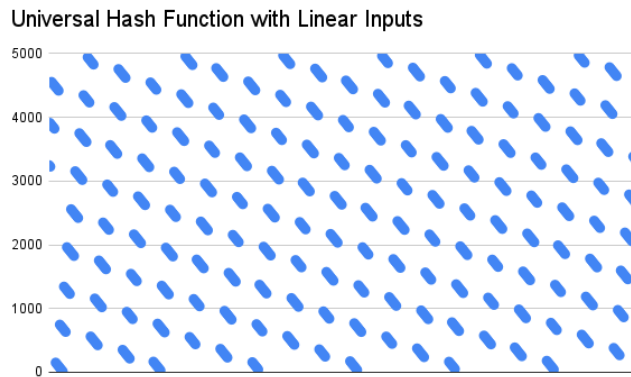


Figure 1: Scatter plot of universal hash function with linear inputs

3.2 Bloom Filter

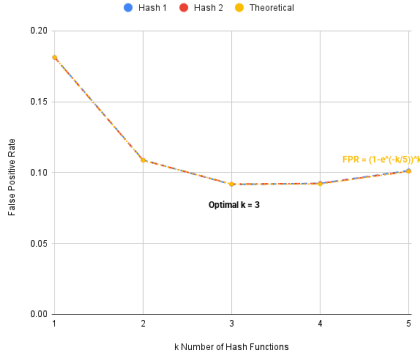


Figure 2:
False Positive Rate for Hash 1 and
Hash 2 versus Theoretical, $c = 5$

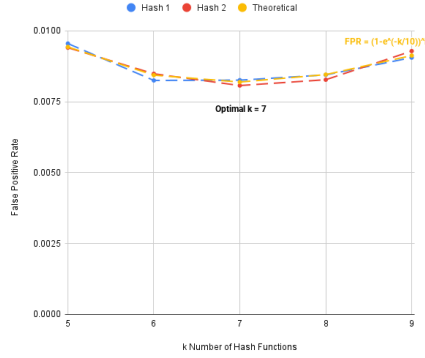


Figure 3:
False Positive Rate for Hash 1 and
Hash 2 versus Theoretical, $c = 10$

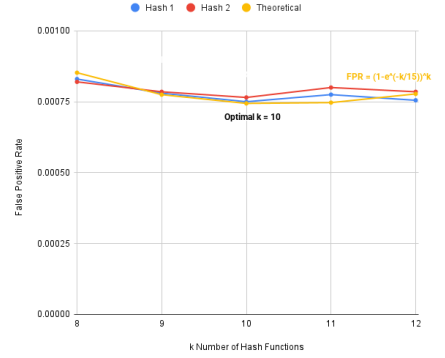


Figure 4:
False Positive Rate for Hash 1 and
Hash 2 versus Theoretical, $c = 15$

Figures 2, 3, and 4 show the plots of false positive rates for $c = 5, 10$, and 15 . For these plots, the theoretical false positive rate for each value of c and k was calculated using the equation $(1 - e^{-\frac{k}{c}})^k$. As the theoretical false positive rate gets smaller, the experimental results deviate more from the theoretical false positive rates. Despite this, for all values of c and k , the theoretical and experimental false positive rates are very close as can be observed by the overlap in the graphs.

4 Conclusions

When hashing linear data, the universal hash produced a scatter plot which indicated correlation. Though interesting, this is not unexpected due to the use of mod and fixed parameters a_i and b_i in the universal hash function. This correlation is likely what caused an average maximum bin load of 1.4. Though a lower average maximum bin load may typically be desirable, namely in chained hashing, an average maximum bin load of 1 would be catastrophic for a bloom filter implementation. With a maximum bin load of 1, every number will hash to a unique bit. With k total hash functions, there will be approximately kn bits set to 1 in this case. If k were equal to c and $m = cn$, there would be a 100% false positive rate since every bit would be set to 1. Therefore, the seeded pseudo-random hash function would perform much better for maintaining a set of correlated inputs, like linear data, in a bloom filter.

In the case of hashing randomly generated values, the results suggest that the two hash functions are nearly equally effective for use in bloom filters. When testing the hash functions individually, both produced similar random-looking scatter plots with no evidence of correlation. Though the slight difference between a 3.2 and 3.5 average maximum bin load may suggest that the universal hash function is marginally better, there is not enough evidence to definitively say that there is a superior hash function. Furthermore, when testing the two types of hash functions in the bloom filter, the false positive rates for both hash functions were very close to the theoretical values. As the false positive rate got smaller, the variance in the experimental data did still cause slight deviation from the theoretical values, but it is far less than would have occurred with a smaller n value. To improve results in the future, a larger n value, such as $n = 1,000,000$ or $n = 10,000,000$ could be selected at the expense of a slower running time. With the results from both experiments considered, I can conclude that the universal hash function and pseudo-random seeded hash function are about equal for maintaining a set of random inputs in a bloom filter.